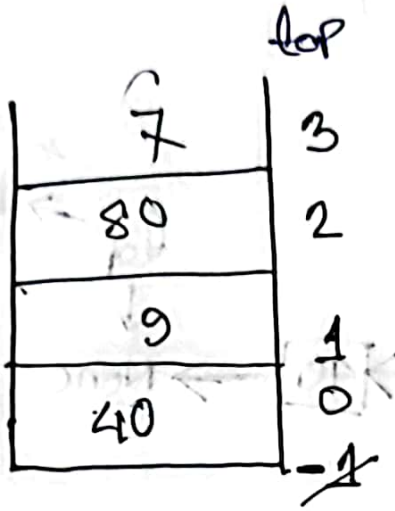
STACK

05-07-23



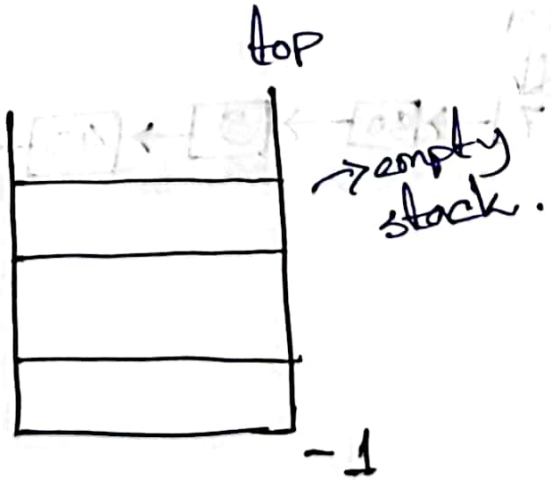
⇒ push (40) → 0
⇒ push (9) → 1
⇒ push (80) → 2
⇒ push (7) → 3

peek ( ) → return 7.

pop ( ) temp = 7 return 7
top -= 1

↳ will remove the last element.

peek ( ) → return 80



pop ( )

pop ( )

pop ( )

peek ( )

↳ stack underflow.

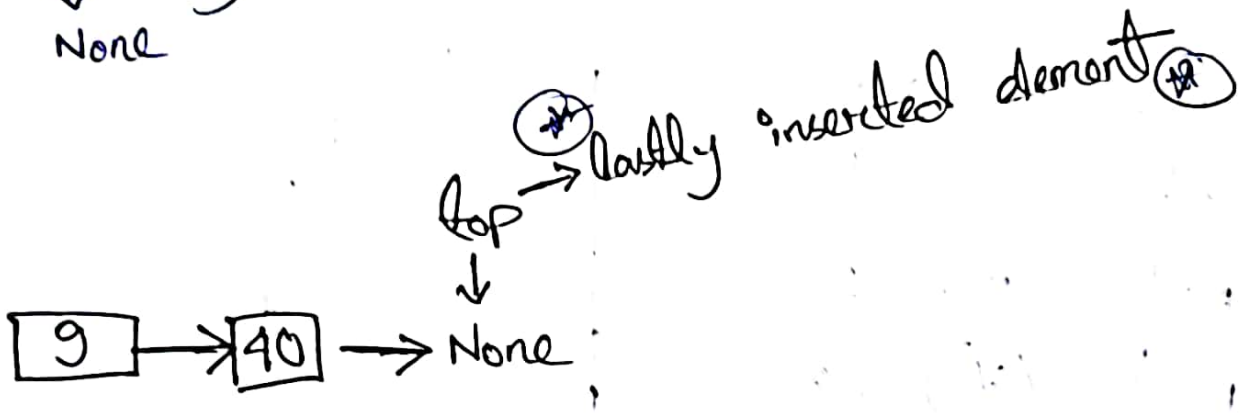
pop ( ).

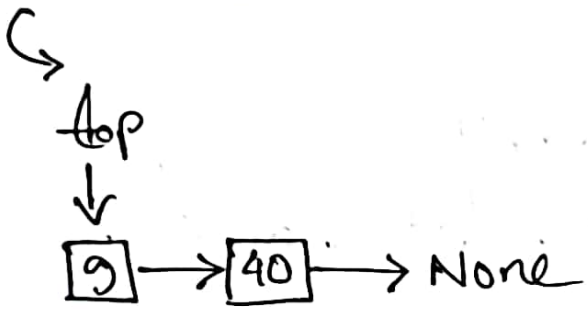
peek ( ) → returns the topmost element.

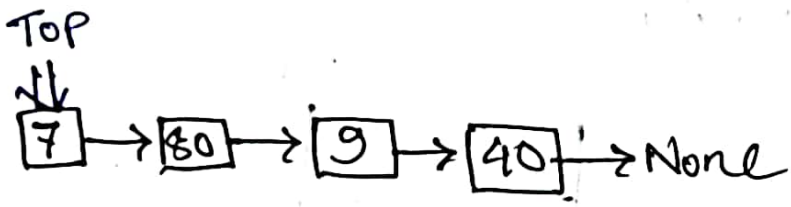
Salicid™

CamScanner

top → None } → means empty stack.







push, pop, peek, isEmpty.

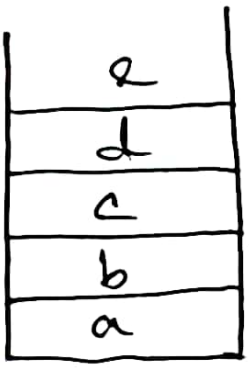
for stack.

Reverse a string using Stack:

```
s = "abcde"
st = stack()

for e in s:
    st.push(e)

result = ''
```



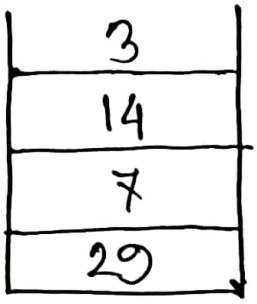
st

```
while st.isEmpty != False: / st.peek() != None:
    v = st.pop()
    result += v
```

Salicid™

CamScanner

(4)



- Remove the last element of stack using only stack operations:

(stack, push, pop, peek, isEmpty)
↳ can use it.

Parenthesis Balancing:

(( ))
({ } [( )]) } every opening parenthesis has a closing one.

1) opening bracket:

(, {, [

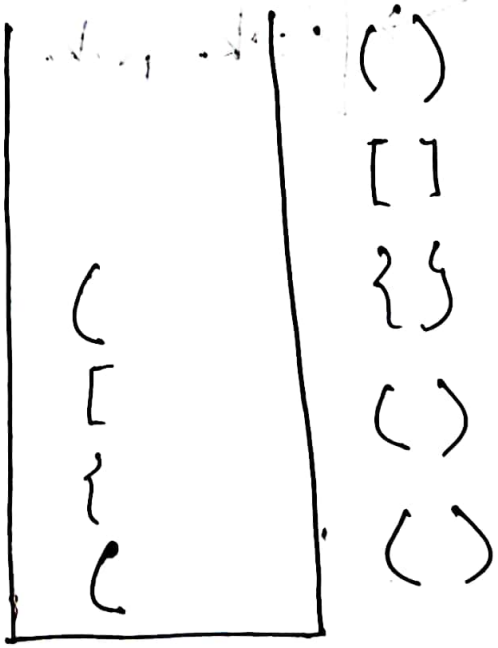
stack.push()

2) closing bracket:

), },]

stack.pop()

④ ({ [( )] } ( ))



Salicid™

CamScanner

correct: string scanning done & stack empty.

Corner Case:

1) mismatch: closing bracket is popped doesn't match type → (}
2) corresponding no opening bracket.

Ex: (( ))]

3) corresponding no closing bracket:

Ex: {(( ))

[string iteration done & stack not empty.]

[stack peek → None/empty → for a closing bracket.]

Code:

```
s = "({}) [()]"
st = stack()
for e in s:
    if e == '(' or e == '[' or e == '{':
        st.push(e)
    elif e == ')' or e == ']' or e == '}':
        v = st.pop()
        if (e == ')' and v != '(') or
           (e == '}' and v != '{') or
           .....
           :
           return False / Break.
```

(1) mismatch

```
    elif v == None:
        return False / break
```

(2)

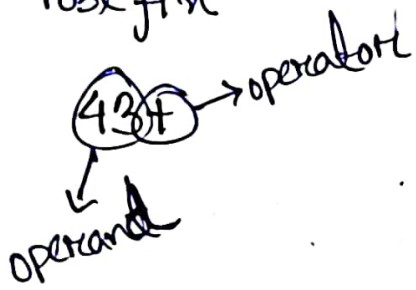
```
a = st.isEmpty().
a → true → Balanced | a → False → InBalanced.
```

Salicid™

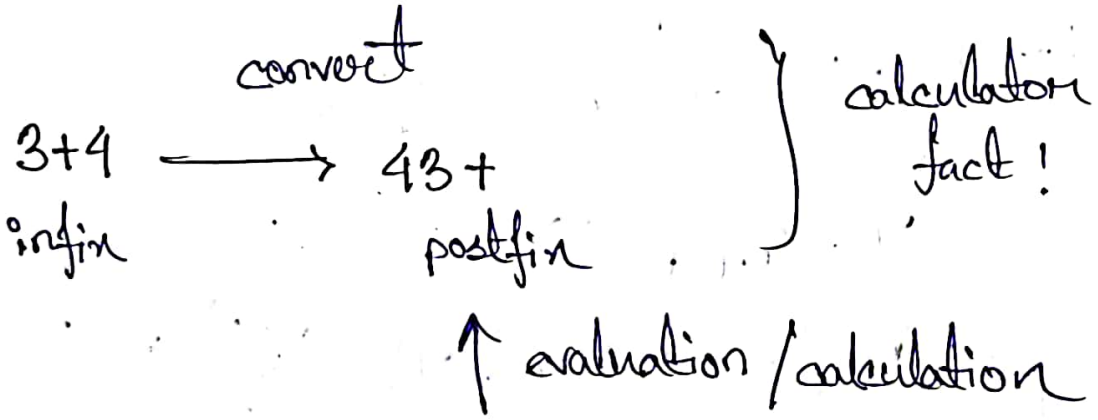
CamScanner

Expression Evaluation:

Infix	Postfix	Prefix
3+4	43+	+43



(ex) 3+4*7



Postfix Expression Evaluation:

Rule:

i) operand → push [at least 2 operand needed]

ii) operator → pop( ) x 2 [for every operand]

iii) calculates / evaluates

iv) push the result.

Ex:

2 7 4 5 * + * 9 / 1 -

character	stack operation	stack	calculation
2	push(2)	2	
7	push(7)	7 2	
4	push(4)	4 7 2	
5	push(5)	5 4 7 2	
*	pop( ) pop( )	7 2	4*5 =20

Salicid™

CamScanner

*	push (20)	20 7 2
+	pop ( ) pop ( )	2
	push (27)	27 2
*	pop( ) pop( )	None
	push (54)	
9	push (9)	9 54
	pop( ) pop ( )	None
	push (6)	6
1	push (1)	1 6
	pop ( ) pop ( )	None
	push (5)	5

7+20 =27
2*27 =54
54/9 =6
6-1 =5

CamScanner

1st pop
2nd operand

2nd pop
1st operand.

Salicid™

CamScanner